Understanding ML, Chapter 5: Learning by Combining Models

After reading this you can explain why averaging helps and when it cannot, what bagging, random forests, boosting, XGBoost, stacking and mixtures of experts each manipulate, which experiment reveals each effect, and what goes wrong in practice

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapters 3 and 4 fitted *one* model and worried about its bias, its variance and its posterior. This chapter fits *many* models and combines them. The reason is the variance term of Chapter 3: a flexible model such as a deep tree reflects the particular sample it was trained on, so a different sample would give a different fit. Averaging several fits cancels the part of the error that differs between them and keeps the part they share. Every method here is a recipe for producing competent models whose errors are as different as possible, plus a rule for combining them. *Voting* combines different algorithms on the same data. *Bagging* gives the same algorithm different bootstrap samples; *random forests* additionally hide features at each split to decorrelate the trees. *Boosting* does something else: it builds models one after another, each one fitted to what the current ensemble still gets wrong, so it attacks the remaining loss (mostly bias) rather than the variance; *XGBoost* is boosting with one regularised second-order objective used for both the tree shape and the leaf values. *Stacking* learns the combination rule from data, and a *mixture of experts* learns a combination that depends on the input. The knobs you manipulate are the number of models M, the correlation between their errors ρ (through bootstrap samples and m_{try}), the tree size, and for boosting the learning rate η, the depth and the number of rounds, with λ and γ as regularisers. Everything you need to compute is on your sheet; what the exam tests is whether you know what each knob does, which experiment shows it, and what breaks.

Contents

1 Why combine models at all **2**

2 Averaging, voting, and how much variance an average removes **4**

3 Decision trees, the base learner **6**

4 Bagging and out-of-bag evaluation **8**

5 Random forests **11**

6 Feature importance **13**

7 Boosting for regression **14**

8 Boosting for binary classification **17**

9 XGBoost: one objective for the tree shape and the leaf values **19**

10 Multi-class boosting, AdaBoost and the libraries **21**

11 Bagging versus boosting, and when ensembles do not help **23**

12 Stacking **24**

13 Mixture of experts **25**

14 Where ensembles are used, and Lab 3 **26**

1 Why combine models at all

Ensemble; competence and diversity

An *ensemble* is a set of M fitted models $\hat{f}_1, \ldots, \hat{f}_M$ whose predictions are combined into one prediction $\hat{f}_{\text{ens}}(x)$ by a vote, an average or a sum of corrections. Two conditions make the combination better than its members. *Competence*: each member must be better than noise, because averaging random guesses gives a random guess. *Diversity*: the members must make *different* errors, because an error that every member makes survives any average. Three identical copies of one model are not an ensemble.

Purpose. Reduce the sample-to-sample variability of a flexible learner (bagging, forests), or reduce the remaining loss step by step (boosting), or let the data decide how to weight different model families (stacking, mixture of experts).

Impact. The prediction at a fixed input becomes more stable (its variance over training samples falls), the decision boundary becomes smoother, and the cost is computation, memory and interpretability.

Instability of one fitted model

Setup. A dataset of N labelled flows (benign or attack), one learning algorithm (say a decision tree grown to purity).

Held fixed. The algorithm and all its settings; the input x_0 at which we look.

What we manipulate. The training sample: draw $B = 50$ different random subsamples (or bootstrap samples) of the data and refit the tree on each.

What we measure. The 50 decision boundaries, and the 50 predicted probabilities at x_0.

What the plot shows. Overlay the 50 boundaries on the two-feature scatter plot: a deep tree gives 50 visibly different staircases; a logistic regression gives 50 nearly identical lines. A histogram of the 50 predictions at x_0 is wide for the tree, narrow for the line.

How to read it. The width of that histogram *is* the variance term $\text{Var}(x_0)$ of Chapter 3. A learner whose boundary jumps around is a candidate for averaging. A learner whose boundary barely moves (large bias, small variance) will not profit from bagging, and only boosting or a richer model can help it.

Three annotators and one ox

At a 1906 livestock fair 787 visitors guessed the weight of an ox; individual guesses were far off, their mean was within one pound of the truth (Galton). The same logic is used to *label* security data: three annotators look at a network event, two say malicious and one says benign; the majority label is kept and strong disagreements go to an expert. Replace annotators by fitted models and you have an ensemble. The annotators are useful only because each knows something and they do not all share the same blind

spot; if all three were trained by the same person with the same checklist, their majority would repeat that checklist's mistakes.

Expecting an ensemble to fix a bias

Tempting because "more models" feels like "more power". But averaging only cancels fluctuations; if every member is missing the same input (no destination port in the features), or every member is the same architecture fitted to the same data, their errors point the same way and the average inherits them. The validation error of the ensemble then sits exactly where the single model's was, and people conclude "ensembles do not work here" when the real problem is a missing feature or a shift between training and deployment. Fix: diagnose the error source first (Section 11).

The three families at a glance

Family	How diversity is obtained	How combined	What is manipulated
Voting / averaging	different algorithms, settings, features or initialisations, usually on the *same* data	fixed vote or average	which models are in the pool
Bagging / random forest	randomised *rows* (bootstrap) and randomised *candidate features* at each split	fixed vote or average	M, tree size, m_{try}
Boosting	each new model targets the *current errors* of the ensemble	successive additive corrections	M, η, depth, λ, γ
Stacking	any pool of base models	a *learned* global weighting	the meta-model
Mixture of experts	experts trained jointly with a gate	a *learned, input-dependent* weighting	gate and routing

Key insight. An ensemble is a device for manufacturing *different* errors and then cancelling them; shared errors are immune to it.

How the professor asks it

Q. *Explain why three copies of the same logistic regression, trained on the same data, form an ensemble that is no better than one copy, and name the two conditions a useful ensemble must satisfy.*
A. No gain, because the three members make *identical* errors, and an average of identical errors is that error. An ensemble helps only when (i) each member is competent, better than chance, and (ii) their errors differ, so that at a given input some members err high and others low and the average cancels part of the fluctuation. Formally the variance of the average is $\sigma^2(\rho + (1-\rho)/M)$ [on your sheet]; identical members have $\rho = 1$ and the variance is σ^2 whatever M is. Consequence: diversity must be engineered (bootstrap rows, random features, residual targets), it does not come from copying.

Q. *A colleague says: "our intrusion detector misses every attack that comes through the new VPN gateway; let us bag it." What is the issue, and what would you do instead?*
A. Bagging will not help, because the error is *shared*: no feature distinguishes VPN traffic, so every bootstrap tree misses the same attacks and their errors have $\rho \approx 1$ on those cases. Averaging reduces variance, not a systematic blind spot. What to do: add the missing information (gateway identity, VPN flags), check for a distribution shift between training traffic and the new gateway's traffic with a temporal or grouped evaluation, and only then consider an ensemble for the remaining instability.

2 Averaging, voting, and how much variance an average removes

Averaging, hard voting, soft voting

For regression the ensemble prediction is the plain average $\hat{f}_{\text{ens}}(x) = \frac{1}{M}\sum_m \hat{f}_m(x)$. For classification, *hard voting* lets each model output a class and keeps the most frequent class; *soft voting* averages the models' posterior estimates, $\hat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M}\sum_m \hat{p}_m(y = c \mid x)$ [on your sheet], and takes the largest average. Hard voting counts heads; soft voting pools *confidence*: a model saying 0.49 and one saying 0.05 cast the same hard vote for class 0 but contribute very differently to a soft vote.

Purpose. Turn several predictions into one, either robustly (hard) or using the margin information (soft).

Impact. Soft voting uses strictly more information, the distance of each probability from the threshold, but only helps if those probabilities are comparable and reasonably calibrated under the same class prior; hard voting caps every member at one vote and is therefore robust to one overconfident member at the price of discarding margins.

Three flow classifiers, one flow

model	$\hat{p}(y = 0 \mid x)$	$\hat{p}(y = 1 \mid x)$	hard vote
A	0.51	0.49	0
B	0.52	0.48	0
C	0.05	0.95	1

Hard vote: class 0, two to one. Soft vote: average of the attack probabilities $(0.49 + 0.48 + 0.95)/3 = 0.64 > 0.5$, class 1. The soft vote used the fact that A and B were coin flips while C was nearly certain. Whether that is the right call depends on whether C's 0.95 deserves belief. If C is a model that says 0.95 whenever it leans towards attack, it will decide every soft vote alone, and the fix is to calibrate it, to weight the models on validation data (stacking), or to fall back to hard voting.

The variance of an average of correlated errors

Fix an input x and let $e_m = \hat{f}_m(x) - f(x)$ be the error of model m, a random variable over training samples. If every error has variance σ^2 and every pair has correlation ρ, the error of the average has

$$\operatorname{Var}(\bar{e}) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big) \quad \text{[on your sheet]}$$

σ^2 is how wrong one member is on its own; ρ is how much of the error *direction* the members share; M is how many you average.

Purpose. It tells you exactly what averaging can and cannot do: it removes the uncorrelated part $(1-\rho)\sigma^2$ at rate $1/M$ and leaves the floor $\rho\sigma^2$ untouched.

Impact. More models push the variance down towards $\rho\sigma^2$ and then stop helping; lowering ρ (bootstrap rows, random features) lowers the floor itself; the formula says nothing about bias.

Deriving $\operatorname{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$

Line 1. The variance of a sum is the sum of all variances plus all pairwise covariances:

$$\operatorname{Var}\Big(\sum_{m=1}^{M} e_m\Big) = \sum_m \operatorname{Var}(e_m) + \sum_{m \neq m'} \operatorname{Cov}(e_m, e_{m'}).$$

Line 2. There are M variance terms, each σ^2, and $M(M-1)$ ordered pairs, each with covariance $\rho\sigma^2$ (correlation times the two standard deviations): the sum is $M\sigma^2 + M(M-1)\rho\sigma^2$.

Line 3. The average divides the sum by M, so its variance divides by M^2:

$$\mathrm{Var}(\bar{e}) = \frac{M\sigma^2 + M(M-1)\rho\sigma^2}{M^2} = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M}.$$

Line 4. Write $(M-1)/M = 1 - 1/M$: $\mathrm{Var}(\bar{e}) = \sigma^2(\frac{1}{M} + \rho - \frac{\rho}{M}) = \sigma^2(\rho + \frac{1-\rho}{M})$.
Why the floor. The M diagonal terms are divided by M^2 and vanish like $1/M$; the $M(M-1) \approx M^2$ cross terms are also divided by M^2 and do *not* vanish. The shared part of the error survives because there are as many cross terms as there are pairs. Special cases: $\rho = 0$ gives σ^2/M; $\rho = 1$ gives σ^2; $M \to \infty$ gives $\rho\sigma^2$.

Variance reduction against M for several correlations

Setup. Simulate: at a fixed x, draw M errors with common variance $\sigma^2 = 1$ and pairwise correlation ρ (or, with real data, fit M models on M bootstrap samples, repeat the whole thing B times, and compute the variance of the averaged prediction across the B repetitions).

Held fixed. σ^2, the input x, the base learner.

What we manipulate. $M \in \{1, 5, 10, 25, 100, 1000\}$ and $\rho \in \{0, 0.2, 0.5, 0.9\}$.

What we measure. The variance of the averaged error, in units of σ^2.

What the plot shows. One curve per ρ, M on a log axis. Recomputed values:

ρ	$M = 1$	5	10	25	100	1000
0	1	0.20	0.10	0.04	0.01	0.001
0.2	1	0.36	0.28	0.232	0.208	0.201
0.5	1	0.60	0.55	0.52	0.505	0.500
0.9	1	0.92	0.91	0.904	0.901	0.900

How to read it. Every curve falls steeply at first and flattens at $\rho\sigma^2$. With $\rho = 0.2$ and $M = 25$ the variance is $0.232\sigma^2$, not the $0.04\sigma^2$ that independence would give; going from 25 to 1000 trees buys only $0.031\sigma^2$. With $\rho = 0.9$ ten models buy almost nothing. The lesson: once the curve is flat, more models are wasted; the only remaining lever is ρ.

Reading σ^2/M when the errors are correlated

People quote "averaging M models divides the variance by M". That is the $\rho = 0$ case, and fitted models trained on overlapping data are never independent. The mistake produces an over-optimistic expectation of how many trees are needed and a mis-diagnosis when adding trees stops helping ("the implementation must be wrong"). Also: the formula is about variance only; averaging does not touch bias, so if every model is missing the same feature, ρ is close to 1 and the ensemble repeats the error. Fix: plot OOB or validation error against M and stop at the plateau; attack the plateau by decorrelating.

Hard voting versus soft voting

	Hard voting	Soft voting
combines	predicted classes	posterior estimates
uses	only which side of the threshold each model is on	the margin (distance from the threshold)
assumption	none beyond competence	probabilities comparable and reasonably calibrated, same prior
failure mode	discards a confident, correct minority	one overconfident model dominates every vote
prefer when	members are miscalibrated or heterogeneous	members are calibrated (e.g. bagged trees of one kind)

Key insight. Averaging buys $(1-\rho)\sigma^2$ and nothing more; every ensemble trick is a way to lower ρ without raising σ^2 too much.

How the professor asks it

Q. *Ten regressors each have error variance $\sigma^2 = 4$ at some input. Compute the variance of their average for $\rho = 0$, 0.3 and 0.9, the limit as $M \to \infty$ at $\rho = 0.3$, and the smallest M that brings the variance to 1.6 or below at $\rho = 0.3$.*
A. Use $\text{Var} = 4(\rho + (1-\rho)/10)$. $\rho = 0$: 0.4. $\rho = 0.3$: $4(0.3 + 0.07) = 1.48$. $\rho = 0.9$: $4(0.9 + 0.01) = 3.64$, barely better than one model. Limit at $\rho = 0.3$: $4\rho = 1.2$. $4(0.3 + 0.7/M) \le 1.6 \iff 0.7/M \le 0.1 \iff M \ge 7$. At $\rho = 0.9$ ten models are nearly useless, so the only productive move is to lower ρ, which is what bootstrap samples and random feature subsets do.

Q. *Give a situation in which hard voting beats soft voting, and one in which soft voting beats hard voting, and say what each rule throws away.*
A. Hard wins when one member is overconfident: two calibrated models at 0.55 and 0.60 for attack and an overconfident model at 0.02 give a soft average 0.39 (benign, wrong if the two are right) but a hard vote 2 to 1 for attack. Soft wins when confidence is deserved: two models at 0.51 for attack (wrong) and one calibrated model at 0.01 give hard 2 to 1 for attack, soft $(0.51 + 0.51 + 0.01)/3 = 0.343$, benign and right. Hard throws away the margins; soft throws away robustness to miscalibration. Four cases on a slide are not a validation set, so the choice is made by validating the two rules.

Q. *Explain why an ensemble of* 1000 *bagged trees can have a validation error almost identical to one of* 50 *bagged trees.*
A. Because the variance of the average has a floor: $\sigma^2(\rho + (1-\rho)/M)$ tends to $\rho\sigma^2$, and by $M = 50$ the term $(1-\rho)\sigma^2/M$ is already small compared with $\rho\sigma^2$. With $\rho = 0.5$, $M = 50$ gives $0.51\sigma^2$ and $M = 1000$ gives $0.5005\sigma^2$: the OOB curve is flat. More trees stabilise, they do not add capacity. To go lower, decorrelate the trees (random forest) or change the error source (boosting, better features).

3 Decision trees, the base learner

Decision tree

A tree with J leaves partitions the feature space into disjoint axis-aligned boxes $S_1, \dots, S_J$ and predicts one constant per box, $T(x) = \sum_j \theta_j \mathbf{1}[x \in S_j]$ [on your sheet]; exactly one indicator is 1, so $T(x)$ is "the constant of the box x falls in". Each box is reached by binary tests $x_j \le t$. The *depth* of a leaf is the number of tests on its path; maximum depth and minimum leaf size are the usual size controls. The ideal fit would choose boxes and constants jointly, with a penalty λJ on the number of leaves; this search is infeasible, so two simplifications are made: for fixed boxes the best constant per leaf has a closed form, and the boxes are chosen *greedily* one split at a time.

Purpose. A flexible, fast, scale-free learner that captures nonlinearity and interactions (a test on x_1 followed by a test on x_2 *is* an interaction), needing no feature scaling because only the order of values matters.

Impact. Depth up: more boxes, lower training error, higher variance. Minimum leaf size up: coarser boxes, smoother, lower variance. A single deep tree is a high-variance learner, which is exactly why ensembles use it.

The optimal leaf constant

For fixed regions the training loss splits by leaf, because each observation lies in exactly one box:

$$\sum_{i=1}^{N} \ell(y^{(i)}, T(x^{(i)})) = \sum_{j=1}^{J} \sum_{i:\, x^{(i)} \in S_j} \ell(y^{(i)}, \theta_j),$$

so each θ_j is chosen on its own leaf. *Squared error*: $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0$ gives

θ = the leaf mean (the finite-sample version of "squared loss selects the conditional mean"). *Absolute error*: the derivative is $\sum_i \text{sign}(\theta - y^{(i)})$, which changes sign when as many points lie below θ as above, so θ = the leaf median. *Zero–one loss*: predict the class that is wrong on the fewest points, the majority class. [on your sheet](leaf: mean, median, majority).

Impurity, Gini, greedy split

The *impurity* $I(S)$ measures how mixed a node is. For classification the Gini impurity is $I(S) = 1 - \sum_c \hat{p}_c(S)^2$ [on your sheet], where $\hat{p}_c(S)$ is the fraction of class c in the node; $\sum_c \hat{p}_c^2$ is the probability that two observations drawn from the node share a class, so I is the probability they differ: 0 for a pure node, 0.5 for two balanced classes. (Entropy $-\sum_c \hat{p}_c \log_2 \hat{p}_c$ is the other common choice [on your sheet].) For regression the within-node squared error plays the same role. The greedy rule picks the feature and threshold with the largest *size-weighted* impurity reduction $I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$ [on your sheet], then recurses in each child until a stopping rule.

Purpose. Decide where to cut, one cut at a time, without searching over all trees.

Impact. Greedy means best *immediate* reduction: a pure but tiny child counts little because of the size weights, and an early split that looks best now can block a better combination later (no look-ahead).

One split by hand: eight flows sorted by packet count

x	1	2	3	4	5	6	7	8
y	0	0	0	1	0	1	1	1

Parent: four of each class, $I = 1 - 0.25 - 0.25 = 0.50$. Candidate $x \le 3.5$: left $\{0,0,0\}$, $I = 0$; right $\{1,0,1,1,1\}$, $I = 1 - 0.8^2 - 0.2^2 = 0.32$; weighted $\frac{3}{8}(0) + \frac{5}{8}(0.32) = 0.20$; reduction 0.30. Candidate $x \le 4.5$: left $\{0,0,0,1\}$ and right $\{0,1,1,1\}$ both have $I = 1 - 0.75^2 - 0.25^2 = 0.375$; weighted 0.375; reduction 0.125. Greedy chooses $x \le 3.5$: a pure left leaf and the larger immediate reduction, even though $x \le 4.5$ splits the data in half.

Hierarchical lock-in: perturb the data, watch the tree

Setup. The eight flows above, a tree grown to depth 2.

Held fixed. The algorithm, the depth.

What we manipulate. Change one label (flip y at $x = 5$ from 0 to 1) and regrow.

What we measure. The root split and everything below it.

What the plot shows. Before: root $x \le 3.5$, then a sub-split on the right. After the flip, the right child becomes $\{1,1,1,1,1\}$, pure, and the whole lower structure disappears; flip a different point and the root itself may move to $x \le 4.5$, rebuilding every box below.

How to read it. One early decision commits everything underneath it; deeper splits can only refine the sub-region they sit in. That is *hierarchical lock-in*, and it is the mechanism behind a tree's high variance. Bagging repairs it by averaging many trees that locked in differently; boosting repairs it by letting each new shallow tree adjust the boundary anywhere.

Trusting a single deep tree, or scaling features for it

A deep tree with zero training error is reported as "the model"; on validation it is far worse, and a tiny change of the data gives a different tree with a different story. Tempting because trees are readable. The fix is the rest of this chapter: average many (bagging, forest) or sum many shallow ones (boosting). A second, harmless-but-telling mistake: standardising features "for the tree". Trees only use the order of the values, so scaling changes nothing; needing to scale is a sign one is thinking of kNN or a penalised linear model.

Tree versus logistic regression as a base learner

	Deep tree	Logistic regression
boundary	axis-aligned staircase, captures interactions	one hyperplane
bias / variance	low bias, high variance	higher bias, low variance
scaling needed	no	yes for penalties and optimisation
what averaging does	removes much of the variance: ideal bagging member	almost nothing: boundaries already agree

Key insight. A tree is a piecewise-constant function chosen greedily; it is flexible and cheap, and its instability is the raw material that ensembles turn into accuracy.

How the professor asks it

Q. *Prove that, for squared-error loss and fixed leaves, the best constant in each leaf is the mean of the targets in that leaf, and state what it is for absolute and zero–one loss.*
A. The training loss separates by leaf since each point is in one leaf, so minimise $\sum_{i \in S_j} (y^{(i)} - \theta)^2$ alone; its derivative $-2\sum_i (y^{(i)} - \theta)$ vanishes at $\theta = \frac{1}{|S_j|}\sum_i y^{(i)}$, and the objective is a convex parabola, so this is the minimum. Absolute loss: the derivative $\sum_i \operatorname{sign}(\theta - y^{(i)})$ changes sign at the median. Zero–one: the majority class. Consequence: a regression tree's leaf outputs are leaf means, which is why boosting's leaf outputs are mean residuals.

Q. *Node with* 10 *flows,* 5 *attacks and* 5 *benign. A split sends* 4 *flows left (all benign) and* 6 *right (*5 *attacks,* 1 *benign). Compute the Gini reduction and explain why the reduction is size-weighted.*
A. Parent $I = 0.5$. Left $I = 0$. Right $I = 1 - (5/6)^2 - (1/6)^2 = 10/36 = 0.278$. Weighted children $0.4 \cdot 0 + 0.6 \cdot 0.278 = 0.167$; reduction $0.5 - 0.167 = 0.333$. Weighting by child size prevents a split that isolates one pure observation from looking as good as a split that purifies half the data: purity on 5% of the rows is worth 5% of the credit.

Q. *Why does the greedy rule not guarantee the best tree of a given depth? Give the shape of a counterexample.*
A. Greedy chooses the split with the best immediate reduction and never revises it. XOR-like data: two features where neither single split reduces impurity at all ($Y = X_1 X_2$ with ± 1 coins) but a split on X_1 followed by a split on X_2 in each child is perfect. The greedy rule sees two useless first splits, picks one at random or stops, and misses the depth-2 tree that is perfect. Consequence: single trees can be far from optimal; ensembles partly compensate by trying many different first splits.

4 Bagging and out-of-bag evaluation

Bootstrap sample; bagging

A *bootstrap sample* $\mathcal{D}^*$ of a training set of size N is N draws from it *with replacement*; $\text{count}(i)$ is how many times observation i was drawn, and the observations with $\text{count}(i) = 0$ are *out of bag* for that sample. *Bagging* (bootstrap aggregating) draws M bootstrap samples, fits the same base learner to each, and averages the predictions or the probabilities, $\hat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_m \hat{f}_m(x)$.

Purpose. Give the *same* learner *different* training sets without discarding data: splitting $\mathcal{D}$ into M disjoint parts would leave each model N/M points, too few.

Impact. Each model sees a perturbed copy of the whole dataset (some points repeated, about a third missing), so the fitted models differ, ρ drops below 1, and the average is more stable than any member; the bias of the base learner is unchanged.

One bootstrap sample of eight flows

From $\{1,\dots,8\}$ eight draws with replacement might give $\{2,5,2,8,1,5,7,2\}$:

observation i	1	2	3	4	5	6	7	8
count(i)	1	3	0	0	2	0	1	1

Flow 2 is seen three times and 5 twice; flows 3, 4 and 6 are out of bag and never influence this tree. A tree fitted to these counts weights flow 2 three times as heavily as flow 1, which is why it differs from a tree fitted to another sample.

The out-of-bag fraction $(1-1/N)^N \to e^{-1}$

Line 1. One draw picks observation i with probability $1/N$, so misses it with probability $1-1/N$. **Line 2.** The N draws are independent, so i is missed by all of them with probability $(1-1/N)^N$ [on your sheet]. **Line 3.** $\lim_{N\to\infty}(1+x/N)^N = e^x$ with $x=-1$ gives $e^{-1}\approx 0.368$. Recomputed: $N=8$: $(7/8)^8 = 0.344$; $N=20$: 0.358; $N=100$: 0.366; $N=1000$: 0.3677. **Consequence.** A large bootstrap sample contains about 63.2% of the distinct observations and leaves 36.8% out, a *different* third for every model. Expected number of models that never saw observation i: $0.368M$.

Out-of-bag prediction and OOB error

For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the models whose bootstrap sample omitted i, about $0.368M$ of them. The OOB prediction uses only those models, $\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|}\sum_{m\in\mathcal{M}_i}\hat{f}_m(x^{(i)})$ [on your sheet](average of probabilities, or a vote, for classification). The *OOB error* is the loss of these predictions over all training observations.

Purpose. A validation estimate *without holding anything out*: for each point, the models that vote on it never saw it, so the prediction is a genuine unseen-data prediction.

Impact. It is free, uses all the data for training, and is a slight underestimate of the full ensemble's quality (only about a third of the trees vote for each point). It estimates the error of a fixed ensemble under the training distribution; it is not a test set and says nothing about deployment shift.

OOB error against the number of trees

Setup. The flow dataset, bagged trees of a fixed size.

Held fixed. The base learner and its size setting ξ (depth, minimum leaf); the data.

What we manipulate. M from 1 to 500, adding trees one at a time and recomputing the OOB error after each.

What we measure. OOB error (and, on a separate held-out validation set, the validation error, to compare).

What the plot shows. A noisy, steeply falling curve that flattens: with $M=1$ almost no point has an OOB vote, so the first values are erratic; by $M\approx 50$ to 200 the curve is flat. The validation curve has the same shape, slightly below the OOB curve.

How to read it. The plateau height is the floor $\rho\sigma^2$ expressed as an error; its position tells you how many trees are enough; beyond it trees cost time and memory but change nothing. The gap between OOB and validation is the OOB pessimism (each OOB prediction uses $0.368M$ trees, not M). If validation lies far *above* OOB, the validation data differ from the training data: a shift, not an OOB defect.

Tuning the tree size at ensemble level

Setup. Bagged trees.

Held fixed. M large enough to be on the plateau (say 300); the data.

What we manipulate. One shared size setting ξ for *all* trees: minimum leaf size $\in\{1,5,20,100\}$ or maximum depth $\in\{2,4,8,\infty\}$.

What we measure. OOB error of the *complete* ensemble for each ξ.

What the plot shows. Typically a U: very small leaves (deep trees) give the best or near-best OOB error because averaging removes their variance; very large leaves underfit. With noisy labels or little data the minimum moves towards larger leaves.

How to read it. The setting is chosen for the *average*, not for any single tree. Tuning each tree separately would optimise the wrong target (one tree's accuracy) and push every tree towards the same strong, correlated fit.

Treating the OOB error as a test result, or tuning on it a hundred times

The OOB error is an internal validation estimate. Two misuses. (i) Reporting it as the final performance: it is tied to the training distribution and cannot detect that deployment traffic differs; keep a temporal or grouped test set untouched. (ii) Selecting tree size, m_{try}, feature set and threshold by repeated OOB comparisons, then reporting the best OOB value: the minimum of many noisy estimates is optimistically biased (the winner's curse of Chapter 3), exactly like reusing a validation set. The number reported is inflated; the fix is a final untouched test set.

OOB versus out-of-fold versus validation set versus test set

	Where it comes from	What it estimates	Limits
OOB	bootstrap by-product inside a bagged ensemble	error of the fixed ensemble under the training distribution	bagging only; slightly pessimistic; overfits if reused for selection
out-of-fold (CV)	explicit K-fold split, works for any learner	error of the *procedure*, averaged over folds	K fits per configuration; folds must respect groups and time
validation set	held-out part of development data	error of one fitted model, used to select configurations	worn out by repeated selection
test set	untouched until the end	performance of the final chosen model	used once; must match deployment conditions

Key insight. Bootstrap sampling manufactures diversity and, as a by-product, a free held-out third of the data for every tree; neither replaces the untouched test set.

How the professor asks it

Q. *Show that a given observation is absent from a bootstrap sample of size N with probability $(1-1/N)^N$, give its limit, and explain what it means for out-of-bag evaluation.*
A. Each of the N independent draws misses the observation with probability $1-1/N$, so all N miss it with probability $(1-1/N)^N \to e^{-1} = 0.368$. So each tree is trained on about 63% of the distinct rows and about 37% of the rows are a held-out set for that tree. For observation i about $0.368M$ trees never saw it; their averaged prediction at $x^{(i)}$ is an honest unseen-data prediction, and averaging the loss over all i gives the OOB error without any explicit split.

Q. *$N = 1000$ flows, $M = 500$ bagged trees. How many distinct flows does a tree see on average, how many trees are OOB for a given flow, and what is the probability that some flow has no OOB prediction at all for $M = 500$ and for $M = 10$?*
A. Distinct per tree $\approx 0.632 \times 1000 = 632$. OOB trees per flow $\approx 0.368 \times 500 = 184$. No OOB prediction means the flow was in-bag for every tree: $0.632^{500} \approx 10^{-100}$, impossible; for $M = 10$, $0.632^{10} = 0.010$, so about 10 of the 1000 flows would have no OOB vote and the OOB error with ten trees is both noisy and incomplete. Consequence: OOB estimates need a reasonably large M.

Q. *Why is the tree-size setting of a bagged ensemble tuned on the OOB error of the whole ensemble rather than tree by tree?*
A. Because the quantity to optimise is the error of the average, not of a member. One tree tuned alone picks a deep, strong fit on all the data; every tree tuned that way converges to the same fit, raising ρ

and recreating the correlated ensemble that bagging was meant to avoid. Procedure: one shared ξ for all $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$, fit a complete ensemble per candidate ξ, compare OOB errors, keep the best, then confirm on an untouched test set.

5 Random forests

Random forest; m_{try}

A random forest is bagging plus one extra randomisation: at *every node* of every tree, only m_{try} of the d features, drawn uniformly at random, are offered as split candidates; the best split among those is taken; a new subset is drawn at every node. Common starting values: $m_{\text{try}} \approx \sqrt{d}$ for classification, $d/3$ for regression. $m_{\text{try}} = d$ recovers bagged trees. Aggregation is by averaging probabilities or voting.

Purpose. Bagged trees can remain too similar: a very strong feature wins the root of almost every bootstrap tree, so the trees have similar shapes and correlated errors (large ρ) and more trees stop helping early. Hiding features forces trees to start differently.

Impact. Small m_{try}: at many nodes the strongest feature is not on offer, so alternatives are used, trees differ, ρ falls; each tree is a little weaker (σ^2 rises a little). The strong feature is not lost, it is available at a fraction m_{try}/d of nodes and is used deeper down. Too small m_{try} gives poor splits (pure noise candidates). More trees M stabilise the forest but do not deepen it.

Probability that a feature is offered

A particular feature is among m_{try} candidates drawn without replacement from d with probability m_{try}/d [on your sheet](by symmetry, or $\binom{d-1}{m_{\text{try}}-1}/\binom{d}{m_{\text{try}}} = m_{\text{try}}/d$; with $d = 25$, $m_{\text{try}} = 5$: $\binom{24}{4}/\binom{25}{5} = 0.2$). It is absent from the root of one tree with probability $1 - m_{\text{try}}/d = 0.8$, and, roots being drawn independently, from all $M = 10$ roots with probability $0.8^{10} = 0.107$ [on your sheet]. Expected roots that offer it: $10 \times 0.2 = 2$, whereas bagging would offer it at all ten and use it at most of them.

The m_{try} sweep: single-tree error against forest error

Setup. Flow data with $d = 25$ features, forests of $M = 300$ trees.

Held fixed. M, leaf size, the bootstrap seeds, the data.

What we manipulate. $m_{\text{try}} \in \{1, 2, 5, 10, 25\}$.

What we measure. Two curves: the average OOB error of the *individual* trees (strength) and the OOB error of the *forest*.

What the plot shows. The single-tree curve is best at $m_{\text{try}} = 25$ (every tree sees the strongest feature) and worsens as m_{try} falls. The forest curve is U-shaped: it improves as m_{try} falls from 25 to around $\sqrt{d} = 5$, because ρ drops faster than σ^2 rises, then worsens at $m_{\text{try}} = 1$ where splits are chosen among pure-noise candidates.

How to read it. The two curves move in opposite directions over the useful range: *weaker trees, stronger forest.* This is $\sigma^2(\rho + (1-\rho)/M)$ in action: with M large the forest variance is essentially $\rho\sigma^2$, and a modest rise in σ^2 is paid back by a large fall in ρ. If signal is carried by very few features (say 5 of 400), $\sqrt{d} = 20$ candidates see no informative feature in $0.9875^{20} = 78\%$ of splits and the minimum moves to larger m_{try}; $\sqrt{d}$ is a starting point, not a law.

Intrusion detection with one dominant feature

In flow data the destination port is often the strongest single feature (port 22 brute force, port 445 worms). Bagged trees all put "port = 22?" at the root and share the mistakes that a port-based rule makes on attacks that use common ports. A forest with $m_{\text{try}} = 5$ of 25 offers the port at only a fifth of the roots; the other trees start with duration, packet rate or byte ratios and catch attacks the port rule misses.

Validation: the forest's false-negative rate on the "attacks on port 443" subgroup drops although each tree alone is slightly worse overall.

Tuning each tree, or adding trees to add capacity

Two confusions. (i) "My trees are weak, let me give them all features": $m_{\text{try}} = d$ makes each tree stronger and the forest worse, because the forest's error is the error of the average. (ii) "The forest underfits, add trees": M only stabilises; once the OOB curve has plateaued, 1000 trees give the same fit as 200. Capacity lives in tree depth, leaf size and m_{try}. The inflated number here is the budget, not an error estimate: hours of training for nothing.

Bagged trees versus random forest

	Bagged trees	Random forest
randomisation	rows (bootstrap)	rows and candidate features at each node
dominant feature	at the root of nearly every tree	at a fraction m_{try}/d of nodes
single tree	stronger, σ^2 small	slightly weaker, σ^2 larger
correlation ρ	large	small
forest variance $\approx \rho\sigma^2$	higher	lower for moderate m_{try}
controls	M, tree size	M, tree size, m_{try}

Key insight. A random forest deliberately weakens each tree to decorrelate the trees; the target is the average's error, and ρ matters more than σ^2 once M is large.

How the professor asks it

Q. *$d = 25$, $m_{\text{try}} = 5$, $M = 10$. What is the probability that x_1 is available at a given split, and that it is unavailable at the roots of all ten trees? Why can hiding x_1 from most roots lower each tree's accuracy and yet improve the forest?*
A. Available with probability $5/25 = 0.2$; unavailable at one root with 0.8, at all ten with $0.8^{10} = 0.107$; about two roots use it. Each tree is weaker (σ^2 up a little) but trees that all start with x_1 make correlated mistakes, and the floor $\rho\sigma^2$ cannot be averaged away; trees that start differently partition the space differently, so ρ falls and the forest variance $\sigma^2(\rho + (1-\rho)/M)$ falls with it. Weaker trees, stronger forest.

Q. *How would you choose m_{try} and the tree size for a forest, and why not tune them separately for each tree?*
A. Start at $m_{\text{try}} \approx \sqrt{d}$ and one shared leaf size; for each candidate pair on a small grid fit a complete forest and compute its OOB error (or a validation error); keep the best; raise M until the OOB error plateaus; keep a test set untouched because repeated OOB selection is optimistic. Not per tree, because the objective is the forest's error: a tree tuned alone picks $m_{\text{try}} = d$ and a deep fit, every tree does the same, and the correlated bagged forest is back.

Q. *A forest's OOB error has been flat from $M = 150$ onwards. The team proposes $M = 2000$ "to be safe". Evaluate.*
A. Useless and costly. The OOB curve measures $\sigma^2(\rho + (1-\rho)/M)$ as an error; past the plateau the $(1-\rho)/M$ term is negligible and only the floor $\rho\sigma^2$ remains, which M cannot touch. Prediction time and memory grow linearly in M for no accuracy. To lower the floor change m_{try}, the leaf size, or the features; to lower bias change the learner (boosting).

6 Feature importance

Mean decrease in impurity (MDI)

Each split at a node t holding N_t of the N training observations earns a *credit* equal to its impurity reduction weighted by the share of data it acts on, $\Delta_t = \frac{N_t}{N}\left[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)\right]$ [on your sheet], credited to the feature used at t. MDI_j sums the credits of all splits on x_j and averages over trees [on your sheet], usually normalised so the MDI values sum to 1: "this feature produced 75% of all the impurity the forest removed".

Purpose. A free, training-time summary of which features the forest *used*.

Impact. Biased towards features with many possible split points (continuous features, high-cardinality categoricals), which get more chances to find *some* reduction by signal or by noise; measures the training fit, not generalisation.

Credits and forest-level MDI

Root with 10 flows (5 attacks, 5 benign), Gini 0.5; the test "destination port = 22?" sends 4 benign flows left (Gini 0) and 6 flows right (5 attacks, 1 benign, Gini $1 - (5/6)^2 - (1/6)^2 = 0.278$). Weighted children $0.4(0) + 0.6(0.278) = 0.167$; with $N_t/N = 1$, $\Delta_t = 0.5 - 0.167 = 0.333$, credited to destination port. Two trees: tree 1 credits port 0.18, duration 0.06; tree 2 credits port 0.12, packets 0.04. Averages over the two trees (absent feature contributes 0): port 0.15, duration 0.03, packets 0.02, total 0.20; normalised MDI = 0.75, 0.15, 0.10.

Permutation importance

For a fitted forest and a feature j: compute the OOB (or validation) error; shuffle the values of x_j among the OOB observations, keeping every other column and the labels fixed; predict again. The importance is the increase in error, $\mathrm{PI}_j = \mathrm{err}(\text{shuffled } x_j) - \mathrm{err}$ [on your sheet]. No tree is retrained; in a forest each tree uses its own OOB sample and the increases are averaged.

Purpose. Measure how much the *fitted model relies* on x_j when predicting, on data it did not train on.

Impact. Shuffling destroys the information in x_j (its link with the label and the other columns) while preserving its marginal distribution, so the trees still receive plausible values. If the error barely moves the forest was not relying on x_j; if it jumps, it was.

The permutation protocol, with its control

Setup. A fitted forest, 100 OOB flows, original OOB error rate 0.10 (10 errors).

Held fixed. The fitted trees (never retrained), the labels, all columns but one.

What we manipulate. Which column is shuffled: destination port, flow duration, total packets; repeat each shuffle $R = 20$ times with different permutations and average, because one shuffle is noisy.

What we measure. Error after shuffling. Recomputed: port 18 errors (0.18, increase 0.08); duration 13 (0.13, +0.03); packets 11 (0.11, +0.01).

What the plot shows. A bar per feature with an error bar from the R repetitions. A useful *control*: add a column of pure noise; its bar should sit at 0 within the error bar, and any real feature whose bar is not clearly above the noise bar is not demonstrably used.

How to read it. Destination port matters most *for this fitted forest*: breaking it adds eight errors. The protocol says nothing about a feature the forest never learned to use because a correlated twin carried the same information.

Reading importance as causality, and missing masked features

Two readings that neither score supports. (i) *Masking*: if x_a and x_b carry the same information, shuffling x_a alone hurts little because trees that use x_b still have it; both look unimportant although the information

they share is essential. Fix: permute them jointly, or drop a column and refit. (ii) *Causality*: a high score means the fitted model relies on the feature; it may be a proxy (destination port stands in for the attacked service), a consequence recorded after the fact ("ticket closed as intrusion": leakage), or a collection artefact. Intervening on it would change nothing. A third slip: ranking features by MDI and believing the continuous ones are "more important" when they merely had more split points.

MDI versus permutation importance

	MDI	Permutation
computed from	training-time split credits	OOB or validation predictions after shuffling
cost	free	one pass per feature per repetition
measures	impurity removed while fitting	reliance of the fitted model in prediction
bias	favours many split points, high cardinality	masked by correlated features
causal?	no	no

Key insight. Importance scores describe the fitted model's reliance, not the world's mechanism; correlated twins hide each other and MDI rewards features with many thresholds.

How the professor asks it

Q. *Two credits in one tree: the root ($N_t/N = 1$) drops impurity from* 0.50 *to* 0.45*; a deeper node holding* 30% *of the rows drops it from* 0.48 *to* 0.08*. Which split earns more MDI credit, and what does this say about depth?*
A. Root: $1 \times (0.50 - 0.45) = 0.05$. Deeper: $0.30 \times (0.48 - 0.08) = 0.12$. The deeper split earns more: credit is share of data times reduction, so a big drop on fewer rows can beat a small drop on all rows. Depth alone does not decide credit; the product does.

Q. *After shuffling, the error rises by* 0.08 *for* x_a *and* 0.02 *for* x_b*; you know* x_a *and* x_b *are highly correlated. What can and cannot be concluded?*
A. Can: the fitted forest leans on x_a more than on x_b in prediction. Cannot: that x_b is unimportant, because when x_a is shuffled the trees that use x_b keep the shared information, and vice versa, so both bars underestimate the shared contribution. Also cannot: that x_a causes attacks. To see the shared part, permute both jointly or drop one and refit. Consequence: use importance for debugging and feature auditing, not for causal claims.

Q. *Why must permutation importance be computed on OOB or validation rows rather than on in-bag training rows?*
A. On in-bag rows a deep tree has memorised the labels, so its error is near zero before shuffling and the increase measures how memorisation breaks, not how the model generalises using x_j. OOB or validation rows give an unseen-data error before and after, so the increase measures reliance in prediction. Using training rows inflates every importance, most for features used deep in the trees.

7 Boosting for regression

Additive boosted model; learning rate

Boosting keeps the observations fixed and builds models *sequentially*: a first learner makes a baseline prediction, the *residuals* $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$ are computed, a second learner is trained exclusively to predict those residuals, and so on. After m rounds, $F_m(x) = F_{m-1}(x) + \eta\, T_m(x)$ with $F_0 = \bar{y}$ [on your sheet]; unrolled, $F_M = F_0 + \eta \sum_m T_m$, a constant plus a scaled *sum* of trees. $\eta \in (0, 1]$ is the learning rate or shrinkage: $\eta = 1$ applies the full correction, $\eta = 0.1$ a tenth of it.

Purpose. Reduce the remaining training loss one step at a time, each tree acting only where

the current ensemble is still wrong.

Impact. Compare bagging's $\frac{1}{M}\sum_m \hat{f}_m$: there every model predicts the whole target and the average smooths them (variance down); here every tree predicts a *correction* and the corrections are summed (loss, mostly bias, down). Smaller η means smaller steps, more rounds for the same fit, usually slightly better generalisation.

Squared error: minimising the loss over the new tree is fitting the residuals

With $L = \frac{1}{2}\sum_i \left(y^{(i)} - F_m(x^{(i)})\right)^2$ and $F_m = F_{m-1} + \eta T_m$,

$$L = \tfrac{1}{2}\sum_i \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)}} - \eta T_m(x^{(i)})\Big)^2.$$

At round m everything already built is frozen, so $r_m^{(i)}$ is a fixed number; the expression is the least-squares objective of a regression whose *targets* are the residuals and whose *predictor* is ηT_m. In practice T_m is fitted to r_m and η is applied in the update. The general recipe replaces the residual by the *pseudo-residual* $r_m^{(i)} = -\partial\ell/\partial F\big|_{F_{m-1}(x^{(i)})}$ [on your sheet], the direction in which changing the prediction at $x^{(i)}$ decreases the loss fastest; for $\ell = \frac{1}{2}(y-F)^2$ it is $y - F$, so "fit the residuals" and "fit the negative gradient" coincide. For other losses they differ, and the leaf outputs are then chosen to minimise the actual loss (next section).

Gradient boosting for regression, the algorithm

(1) $F_0(x) = \arg\min_w \sum_i \ell(y^{(i)}, w)$, the mean for squared error. (2) For $m = 1, \ldots, M$: (a) pseudo-residuals $r_m^{(i)} = -\partial\ell/\partial F$ at $F_{m-1}(x^{(i)})$; (b) fit a regression tree T_m to the targets $r_m^{(i)}$, giving leaves $S_{1,m}, \ldots, S_{Q_m,m}$; (c) in each leaf output the mean residual $w_{q,m} = \frac{1}{|S_{q,m}|}\sum_{i\in S_{q,m}} r_m^{(i)}$; (d) $F_m = F_{m-1} + \eta T_m$.

Purpose. A loss-agnostic way to add corrections: the tree learns *where* to correct, the leaf values *how much*.

Impact. Training loss decreases monotonically for ever; validation loss decreases and then rises once new trees fit noise.

One round by hand, $\eta = 0.1$

ID	x	y	F_0	$r_1 = y - F_0$	F_1	$r_2 = y - F_1$
1	1.2	10	20	-10	$20 - 0.1\cdot 10 = 19$	-9
2	4.5	20	20	0	$20 + 0.1\cdot 5 = 20.5$	-0.5
3	7.1	30	20	$+10$	20.5	9.5

$F_0 = \text{mean}(10, 20, 30) = 20$. Tree T_1 splits at $x < 2.85$: leaf $\{1\}$ with weight -10, leaf $\{2, 3\}$ with weight $(0 + 10)/2 = 5$. The update moves ID 3 from 20 to 20.5, closer to 30. Squared error $\frac{1}{2}\sum r^2$: 100 before, 85.75 after one shrunken step; the next tree is fitted to $(-9, -0.5, 9.5)$. With $\eta = 1$ the same tree would jump to $F_1 = (10, 25, 25)$ in one go: faster, and more willing to chase noise.

Boosting rounds: training against validation loss, two learning rates

Setup. Regression on house prices or flow byte counts; gradient-boosted trees of depth 3.

Held fixed. Depth, the data split, the loss.

What we manipulate. The number of rounds M from 1 to 2000, for $\eta = 1$ and $\eta = 0.1$.

What we measure. Training loss and validation loss after every round.

What the plot shows. Four curves. Both training curves fall monotonically for ever. Each validation curve falls, reaches a minimum, and rises: for $\eta = 1$ the minimum comes early (tens of rounds) and the

rise is steep; for $\eta = 0.1$ the minimum comes roughly ten times later, is slightly lower, and the rise is gentle.

How to read it. The validation minimum is the early-stopping point M^*. Halving η roughly doubles M^* (the coupling $\eta \propto 1/M$); the smaller rate wins a little accuracy for ten times the computation. The training curve is useless for choosing M: it never turns up.

Depth, rate and rounds interact

Setup. Same data.

Held fixed. Early stopping on validation for every configuration.

What we manipulate. A grid over depth $\in \{1, 2, 4, 8\}$ and $\eta \in \{0.3, 0.1, 0.03\}$; for each cell M is chosen by early stopping.

What we measure. Validation loss at the selected round, and M^*.

What the plot shows. A heat map of validation loss over (depth, η), with M^* printed in each cell: deeper trees reach their minimum in fewer rounds; depth 1 (stumps) can only model additive effects and plateaus higher when interactions matter; depth 4 to 6 usually wins; very deep trees overfit early.

How to read it. The three knobs are coupled and must be tuned together: depth limits the order of interactions a tree can express, η sets the step size, M the number of steps. Comparing configurations at a *fixed* M would be unfair to small η.

Choosing M by training loss, or tuning η and M separately

Training loss keeps falling, so "more rounds" always looks better on it; the validation curve is the only one that turns. Reporting the training curve produces a model that has memorised the noise. Second slip: fixing $M = 100$ and comparing $\eta = 0.3$ with $\eta = 0.01$; the small rate has not converged and looks bad, although at its own M^* it would win. Third: expecting an OOB error from boosting; there is none, the rounds are sequential and every tree sees the full (reweighted) data, so you must hold out a validation set or cross-validate.

One deep tree versus many shallow boosted trees

One deep tree	Boosted shallow trees
fits the whole rule at once	tree 1 isolates the loudest pattern; later trees target leftovers
hierarchical lock-in: early splits force rigid boundaries, deep splits only refine their sub-region	global adaptability: each new tree may adjust the boundary anywhere in feature space
a poor early split cannot be revised and amplifies noise	shrinkage η enforces gradual convergence
low bias, high variance in one shot	each tree high bias, low variance; the sum lowers bias while η and early stopping keep variance in check

Key insight. Boosting is a sum of shrunken corrections, each fitted to the negative gradient of the loss; it buys bias reduction with sequential steps, and its only brake is the validation curve.

How the professor asks it

Q. *Derive that, for squared-error loss, choosing the next tree to minimise the ensemble loss is the same as fitting the tree to the current residuals.*

A. Substitute $F_m = F_{m-1} + \eta T_m$ into $L = \frac{1}{2}\sum_i (y^{(i)} - F_m(x^{(i)}))^2$ and group $y^{(i)} - F_{m-1}(x^{(i)}) = r_m^{(i)}$, a fixed number because F_{m-1} is already fitted: $L = \frac{1}{2}\sum_i (r_m^{(i)} - \eta T_m(x^{(i)}))^2$, the least-squares objective with targets r_m and predictor ηT_m. The negative gradient of $\frac{1}{2}(y - F)^2$ in F is $y - F$, so the residual is also the pseudo-residual; that is why the general algorithm says "fit the negative gradient".

Q. $y = (0, 2, 4, 6)$ *at* $x = (1, 2, 3, 4)$, $F_0 = \bar{y}$, *one stump splitting at* $x \le 2.5$, $\eta = 0.5$. *Compute* F_1, *the new residuals, and the squared error before and after.*
A. $F_0 = 3$, residuals $(-3, -1, 1, 3)$. Left leaf mean $(-3 - 1)/2 = -2$, right $(1 + 3)/2 = 2$. $F_1 = 3 + 0.5 \cdot (-2, -2, 2, 2) = (2, 2, 4, 4)$; residuals $(-2, 0, 0, 2)$. Squared error $\sum r^2$: $9 + 1 + 1 + 9 = 20$ before, $4 + 0 + 0 + 4 = 8$ after. With $\eta = 1$ it would be $(1, 1, 5, 5)$ and residuals $(-1, 1, -1, 1)$, error 4: the full step fits more now and leaves a residual pattern a stump cannot fix, which is the trade shrinkage makes.

Q. *Explain why the learning rate, the number of rounds and the tree depth cannot be tuned one at a time, and describe the procedure you would use.*
A. They are coupled: halving η roughly doubles the number of rounds needed to reach the same fit, and deeper trees reach the validation minimum in fewer rounds, so the best M depends on the other two. Procedure: grid or random search over (depth, η); for each pair run boosting with early stopping on validation (or cross-validation) to pick M^*; compare configurations by their validation loss at their own M^*; confirm the winner on an untouched test set.

8 Boosting for binary classification

Boosting the logit

Probabilities live in $(0, 1)$ and cannot be summed with tree outputs ($0.9+0.4$ is not a probability). So the ensemble produces an unconstrained score $F^{(i)}$ and maps it through the sigmoid, $\hat{p}^{(i)} = \sigma(F^{(i)})$ [on your sheet]; trees are added to F, and $\hat{p}$ is recomputed. The loss is the binary cross-entropy written in the logit, $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$. This is the discriminative route of Chapter 4 with the score $s_\theta(x)$ now a sum of trees instead of $w^\top x + b$.

Purpose. Keep every intermediate ensemble a legal probability model while adding corrections freely.

Impact. The gradient and the curvature of the loss in F are $g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$ [on your sheet]; the pseudo-residual $-g = y - \hat{p}$ is "how far the probability is from the label".

$g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$, line by line

Line 1. $\sigma'(F) = \sigma(F)(1 - \sigma(F))$: with $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1-\sigma)$. **Line 2.** $\partial \ell / \partial \hat{p} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}$. **Line 3 (chain rule).** $g = \frac{\partial \ell}{\partial \hat{p}} \frac{d\hat{p}}{dF} = (-\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}})\hat{p}(1-\hat{p}) = -y(1 - \hat{p}) + (1 - y)\hat{p} = -y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y$. The marks are in this cancellation. **Line 4.** $h = \partial(\hat{p} - y)/\partial F = d\hat{p}/dF = \hat{p}(1-\hat{p})$. **Reading.** h is the variance of a Bernoulli with parameter $\hat{p}$: it peaks at 0.25 when $\hat{p} = 0.5$ (unsure, the loss curves most) and vanishes when $\hat{p} \to 0$ or 1 (confident, the loss is flat in F). For $y = 1$ and a very negative F (confidently wrong), $-g \to 1$ is at its largest while $h \to 0$: a large residual with almost no curvature. Keep this picture.

Gradient boosting for classification and the line-search bottleneck

At round m: (1) pseudo-residuals $r_m^{(i)} = y^{(i)} - \hat{p}_{m-1}^{(i)}$; (2) grow a regression tree on them by squared-error splits (*the split search determines the regions*); (3) with the regions fixed, choose each leaf's output by a *line search* on the true loss, $w_{q,m} = \arg\min_w \sum_{i \in S_{q,m}} \ell(y^{(i)}, F_{m-1}^{(i)} + w)$ (*the line search determines the outputs*); (4) $F_m = F_{m-1} + \eta T_m$.

Purpose. Choose leaf values that minimise the actual log loss, not the squared error of the pseudo-residuals.

Impact. For squared error step 3 was "mean residual"; for log loss the leaf objective is a sum of log-sigmoids in w with no closed form, so it must be minimised numerically leaf by leaf, round after round: the *line-search bottleneck*.

The Newton leaf value, with and without λ

Expand the loss of observation i to second order around its current logit: $\ell(F+w) \approx \ell(F) + gw + \frac{1}{2}hw^2$ [on your sheet]. Summing over a leaf and adding an L_2 penalty $\frac{\lambda}{2}w^2$ gives a quadratic in the single number w, $\sum_i \ell^{(i)} + G_q w + \frac{1}{2}(H_q+\lambda)w^2$ with $G_q = \sum_{i\in q} g^{(i)}$, $H_q = \sum_{i\in q} h^{(i)}$; its leading coefficient is positive since $h > 0$. Derivative $G_q + (H_q+\lambda)w = 0$ gives

$$w_q^* = -\frac{G_q}{H_q+\lambda} = \frac{\sum_{i\in q}(y^{(i)} - \hat{p}^{(i)})}{\sum_{i\in q}\hat{p}^{(i)}(1-\hat{p}^{(i)}) + \lambda} \quad \text{[on your sheet]}$$

and $\lambda = 0$ is the plain Newton step "total residual over total curvature". In a leaf of unsure observations ($h \approx 0.25$ each) the step is moderate; in a leaf of confidently wrong observations ($h \approx 0$) the denominator collapses and the step *explodes*. λ is a floor on the denominator that caps the step, and it matters most exactly where $\sum h$ is smallest.

Two leaves, one safe and one explosive

Leaf A holds $(y, \hat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. $g = \hat{p} - y = -0.8, -0.6, +0.8$, $G = -0.6$; $h = \hat{p}(1-\hat{p}) = 0.16, 0.24, 0.16$, $H = 0.56$. Newton value $-G/H = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$; with $\eta = 0.1$ the logit moves by 0.0385. The positive sign raises all three probabilities, helping the two positives more than it hurts the negative.
Leaf B holds two confidently wrong positives $(1, 0.01), (1, 0.02)$: $G = -1.97$, $H = 0.0099 + 0.0196 = 0.0295$. Newton value $1.97/0.0295 = 66.8$: a logit jump of 67 from two observations, pure overfitting. With $\lambda = 1$: $1.97/1.03 = 1.91$; with $\lambda = 0.1$: 15.2. Minimum child weight (reject a leaf whose H is below a threshold) would refuse to create leaf B at all.

Confusing the gradient with the residual, and ignoring the Hessian

The slides use both $r = y - \hat{p}$ and $g = \hat{p} - y$; they are consistent ($r = -g$) but a sign slip flips the direction of every update. Always ask "gradient or negative gradient?" before reading a formula. Second: treating the Newton target $-g/h$ of a confidently wrong point ($\approx 1/\hat{p}$, huge) as its influence; its weight h is tiny and the product $h \cdot (-g/h) = y - \hat{p}$ is bounded by 1, so the point contributes at most 1 to the numerator and nothing to the denominator. Third: running unregularised Newton steps on small leaves; the outputs explode and the validation log loss diverges while training loss looks wonderful.

Regression boosting versus classification boosting

	Squared error	Log loss
what is boosted	the prediction itself	the logit F, with $\hat{p} = \sigma(F)$
pseudo-residual	$y - F$	$y - \hat{p}$
curvature h	1 for every point	$\hat{p}(1-\hat{p})$, between 0 and 0.25
leaf output	mean residual (closed form)	line search, or Newton $\sum(-g)/(\sum h + \lambda)$
risk	none special	exploding steps where $\sum h \approx 0$

Key insight. In logit space the residual is $y - \hat{p}$ and the curvature is $\hat{p}(1-\hat{p})$; the Newton step divides one by the other, which is why confident leaves need λ.

How the professor asks it

Q. *Prove $g = \hat{p} - y$ and $h = \hat{p}(1-\hat{p})$ from $\ell = -y\ln\hat{p} - (1-y)\ln(1-\hat{p})$ with $\hat{p} = \sigma(F)$, and say why the Newton leaf objective is convex.*
A. Four lines: $\sigma' = \sigma(1-\sigma)$; $\partial\ell/\partial\hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$; multiply by $\hat{p}(1-\hat{p})$ and simplify $-y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y$; differentiate again, $h = d\hat{p}/dF = \hat{p}(1-\hat{p})$. Since $h > 0$ for $\hat{p} \in (0,1)$, the quadratic $G_q w + \frac{1}{2}(H_q+\lambda)w^2$ has a positive leading coefficient, so it has a unique minimiser $-G_q/(H_q+\lambda)$.

Q. *One observation with $y = 1$ has $\hat{p}$ very close to 0. Its Newton target $-g/h$ is enormous. What happens to h, and why must the two be read together?*
A. $-g = 1 - \hat{p} \to 1$, $h = \hat{p}(1-\hat{p}) \to 0$, target $-g/h = 1/\hat{p} \to \infty$: the quadratic model says the loss is flat here, so a unit residual asks for a huge step. But its weight in the leaf objective is the same $h \to 0$, so what enters $w^* = \sum(-g)/(\sum h + \lambda)$ is $h \cdot (-g/h) = 1 - \hat{p} \leq 1$ in the numerator and almost nothing in the denominator. Target alone says "thousands", weight alone says "irrelevant"; the truth is a bounded pull in the right direction. A leaf made *only* of such points has $\sum h \approx 0$: that is what λ and minimum child weight are for.

Q. *Leaf $(1, 0.3), (1, 0.6), (0, 0.2)$. Compute g, h, G, H, the leaf value for $\lambda = 0$ and $\lambda = 1$, and the loss removed at $\lambda = 1$.*
A. $g = -0.7, -0.4, +0.2$, $G = -0.9$; $h = 0.21, 0.24, 0.16$, $H = 0.61$. $w^* = 0.9/0.61 = 1.475$ at $\lambda = 0$, $0.9/1.61 = 0.559$ at $\lambda = 1$. Loss removed $\frac{1}{2}G^2/(H+\lambda) = \frac{1}{2}(0.81)/1.61 = 0.252$. The regularised step is less than half the raw step because H is small compared with $\lambda = 1$; λ bites hardest in low-curvature leaves.

9 XGBoost: one objective for the tree shape and the leaf values

The XGBoost tree objective

A proposed tree with leaves $S_1, \ldots, S_Q$ and outputs w_q is scored by

$$\mathcal{L}_m = \sum_q \sum_{i \in S_q} [g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2] + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2 \quad \text{[on your sheet]}$$

Three parts: the second-order Taylor approximation of the loss change produced by the tree (the constant $\sum_i \ell^{(i)}$ is dropped), a charge γ per leaf, and a charge $\frac{\lambda}{2} w_q^2$ for large outputs.

Purpose. Use one number to score both the *structure* (which observations share a leaf) and the *outputs*, so that gradients, Hessians and regularisation are used *while growing* the tree, not after.

Impact. Compared with the GBM variants, which build the structure by unweighted squared-error splits on $y - \hat{p}$ and only afterwards compute Newton outputs, XGBoost's split search already sees the curvature; it is faster (one pass) and better regularised.

Completing the square: the objective is a weighted regression

For observation i in leaf q, $\frac{1}{2}h(w + g/h)^2 = \frac{1}{2}hw^2 + gw + g^2/(2h)$, hence

$$g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2 = \tfrac{1}{2} h^{(i)} \left(r_{\text{XGB}}^{(i)} - w_q\right)^2 - \frac{(g^{(i)})^2}{2h^{(i)}}, \qquad r_{\text{XGB}}^{(i)} = -\frac{g^{(i)}}{h^{(i)}} = \frac{y^{(i)} - \hat{p}^{(i)}}{\hat{p}^{(i)}(1 - \hat{p}^{(i)})} \quad \text{[on your sheet]}$$

and the dropped term does not involve w. So $\mathcal{L}_m$ is a *weighted least-squares* problem: each observation asks the tree to output r_{XGB}, its own one-step Newton correction, and its request is weighted by h. An unsure observation (h large) has a modest target and a loud voice; a confidently wrong one has a huge target ($\approx 1/\hat{p}$) and a whisper of a weight; their product $h \cdot r_{\text{XGB}} = y - \hat{p}$ is bounded by 1. "XGBoost uses the Hessian in the split search" means the tree is grown by weighted squared-error splits on these targets.

Optimal leaf value and the gain of a split

Summing over a leaf: $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda) w^2$, minimised at $w_q^* = -G_q/(H_q + \lambda)$ with value $\phi_q(w_q^*) = -\frac{1}{2} G_q^2/(H_q + \lambda)$ [on your sheet], "how much loss this leaf can remove with its best output" (more negative is better). Replacing a parent (G, H) by children (G_L, H_L), (G_R, H_R) with $G = G_L + G_R$, $H = H_L + H_R$ changes the data-fit part by $\phi_L + \phi_R - \phi_{\text{parent}}$ and the leaf

count by one, so

$$\text{gain} = \tfrac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\Big] - \gamma \quad \text{[on your sheet]}$$

and the split is accepted iff gain > 0. The two penalties act differently: λ sits inside every denominator and *shrinks* all outputs and all three terms, most strongly where H is small; γ is a flat per-leaf price that *discourages* marginal splits. One line: λ shrinks leaf outputs, γ discourages unnecessary splits.

Does the split pay for its leaf?

Parent $(G, H) = (-4, 3)$, children $(-3, 1)$ and $(-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Parent: $w^* = 4/4 = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 3/2$, $\phi = -\frac{1}{2} \cdot 9/2 = -2.25$. Right: $w^* = 1/3$, $\phi = -\frac{1}{2} \cdot 1/3 = -0.167$. Children together -2.417: the fit improves by 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept. The split would be rejected for any $\gamma \geq 5/12 \approx 0.417$. Note why the children win: the parent mixes $G_L = -3$ with $G_R = -1$, same sign, so the parent's single output already serves both; opposite-sign gradients in one leaf cancel in G and splitting them is where the big gains are.

Sweeping λ and γ at fixed depth and rate

Setup. Flow classification, XGBoost with depth 4, $\eta = 0.1$, early stopping.

Held fixed. Depth, η, subsampling off.

What we manipulate. $\lambda \in \{0, 1, 10, 100\}$ and separately $\gamma \in \{0, 0.1, 1, 10\}$.

What we measure. Validation log loss at the early-stopping round, the average number of leaves per tree, and the distribution of leaf outputs.

What the plot shows. Raising λ compresses the histogram of leaf outputs towards 0 (and removes the extreme values from near-pure leaves) with the number of leaves nearly unchanged; raising γ cuts the number of leaves (trees stop splitting early) with the surviving outputs nearly unchanged. Validation loss is U-shaped in both.

How to read it. The two knobs have different fingerprints: λ is continuous shrinkage of outputs, γ is a discrete tax on structure. Minimum child weight is a third knob with a third fingerprint: it removes leaves whose H is small, exactly the explosive ones.

Mixing up λ, γ and the slides' λJ

The slides use λJ for a penalty on the number of leaves when introducing trees and later λ for the L_2 penalty on leaf outputs; XGBoost's per-leaf price is γ. Students write "λ discourages splits": wrong, λ shrinks outputs (continuous), γ taxes leaves (discrete, exactly γ per binary split since a split adds one leaf). A second slip: believing XGBoost "is" a different algorithm from gradient boosting; it is gradient boosting with a second-order, regularised objective used for structure and outputs, plus engineering (histogram splits, sparse handling, parallel split evaluation, GPU). Rounds remain sequential.

The four leaf-value mechanisms

Algorithm	Structure target (split search)	Leaf value mechanism
Basic boosting	raw error $y - \hat{p}$	exact numerical line search
Unregularised GBM	raw error $y - \hat{p}$	Newton step $\sum(-g)/\sum h$
Regularised GBM	raw error $y - \hat{p}$	regularised step $\sum(-g)/(\sum h + \lambda)$
XGBoost	Newton target $-g/h$, weighted by h	same regularised step, found in the same weighted pass that chose the splits; gain includes γ

The five knobs of boosted trees

Knob	What it manipulates	Fingerprint when raised	Coupled with
M (rounds)	number of corrections	training loss down for ever; validation turns up	η, depth
η	size of each correction	slower, smoother, slightly better at its own M^*	M
depth / leaves	interaction order a tree can express	fewer rounds to the minimum; overfits if deep	M, γ
λ	shrinks every leaf output, most where H small	outputs compressed, leaves unchanged	minimum child weight
γ	price per leaf	fewer leaves, outputs unchanged	depth

Key insight. XGBoost grows the tree with the same regularised quadratic it uses for the outputs: every observation asks for $-g/h$ with weight h, λ shrinks the answer and γ taxes every extra leaf.

How the professor asks it

Q. *Starting from the leaf contribution $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda)w^2$, derive the optimal output, the loss it removes, and the gain of splitting a parent into two children; explain why γ enters as a plain subtraction.*
A. $\phi'(w) = G_q + (H_q + \lambda)w = 0$ gives $w_q^* = -G_q/(H_q + \lambda)$; substituting, $\phi(w^*) = -G_q^2/(H_q + \lambda) + \frac{1}{2}G_q^2/(H_q + \lambda) = -\frac{1}{2}G_q^2/(H_q + \lambda)$. Replacing one leaf by two changes the fit from ϕ_{parent} to $\phi_L + \phi_R$, a decrease of $\frac{1}{2}[G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)]$, and raises the leaf count by exactly one, so the penalty rises by $\gamma(Q + 1) - \gamma Q = \gamma$: a flat threshold the improvement must clear.

Q. *Parent $(G, H) = (-2, 4)$, children $(-2.5, 2)$ and $(0.5, 2)$, $\lambda = 1$, $\gamma = 0.3$. Accept the split? Interpret the signs of G_L and G_R.*
A. Parent $\phi = -\frac{1}{2} \cdot 4/5 = -0.4$. Left $\phi = -\frac{1}{2} \cdot 6.25/3 = -1.042$. Right $\phi = -\frac{1}{2} \cdot 0.25/3 = -0.042$. Children -1.083, improvement 0.683, gain $0.683 - 0.3 = 0.383 > 0$, accept. The parent's $G = -2$ hides a left group pulling strongly one way (-2.5) and a right group pulling the other way $(+0.5)$; in one leaf they partly cancel and the single output $w^* = 0.4$ serves neither well. Separating opposite-sign gradients is exactly where splits earn their keep.

Q. *Explain in one paragraph what "XGBoost uses the Hessian in the split search" means and why the GBM variants do not.*
A. Completing the square turns the second-order objective into a weighted least-squares problem with per-observation target $-g/h$ and weight h; XGBoost evaluates candidate splits by the gain formula, which depends on G and H of each child, so curvature shapes the tree. The GBM variants grow the tree by unweighted squared-error splits on the raw residual $y - \hat{p}$ and use h only afterwards for the frozen leaves' Newton outputs; their split search is blind to curvature. Consequence: XGBoost separates confident from unsure observations when that improves the quadratic objective, and its gain already includes λ and γ.

10 Multi-class boosting, AdaBoost and the libraries

K trees per round in softmax space

For K classes the ensemble keeps a vector of logits $F_1, \ldots, F_K$, maps it to probabilities with the softmax, and uses the multi-class cross-entropy $\ell = -\sum_k y_k \ln \hat{p}_k$ with one-hot y. Per observation, $g_k = \hat{p}_k - y_k$ and $\partial^2 \ell / \partial F_k \partial F_r = \hat{p}_k(\delta_{kr} - \hat{p}_r)$ [on your sheet]: $\hat{p}_k(1 - \hat{p}_k)$ on the diagonal, $-\hat{p}_k \hat{p}_r$ off it. Keeping only the diagonal, the objective becomes a *sum* of K independent binary-style objectives, so one tree is fitted per class per round with target $r_{\text{XGB},k} = (y_k - \hat{p}_k)/(\hat{p}_k(1 - \hat{p}_k))$, formally identical to the binary case.

Purpose. Reuse the binary machinery for any number of classes.

Impact. The softmax couples the classes (raising F_1 lowers every other $\hat{p}_k$); the diagonal approx-

imation ignores that coupling within a round and restores it when the softmax is recomputed from all updated logits at the end of the round. K times the work per round; the K fits can run in parallel, the rounds cannot.

Softmax derivatives

Write $\ell = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$, using $\ln \hat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$. Then $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \hat{p}_k - y_k$. Differentiating $\hat{p}_k = e^{F_k} / \sum_j e^{F_j}$ with respect to F_r by the quotient rule gives $\hat{p}_k(\mathbf{1}[k = r] - \hat{p}_r)$.

AdaBoost versus gradient boosting versus the libraries

Name	Type	Main emphasis
AdaBoost	a boosting *algorithm*	raises the weights of misclassified observations so the next weak classifier focuses on them; the ensemble is a weighted vote of classifiers, not a sum of regression trees
Gradient boosting	a boosting *algorithm*	fits each tree to the negative gradient of a chosen loss; leaf values by line search or Newton
XGBoost	a *library*	second-order regularised objective for splits and outputs; histogram splits, sparsity, parallel split evaluation, GPU
LightGBM	a *library*	histogram-based splits and leaf-wise growth for large datasets
CatBoost	a *library*	ordered treatment of categorical features designed to limit target leakage

Listing AdaBoost, XGBoost, LightGBM and CatBoost as four algorithms of one kind

AdaBoost reweights *observations* and combines *classifiers* by weighted vote; gradient boosting fits *trees to gradients* and sums them. XGBoost, LightGBM and CatBoost are implementations of gradient-boosted trees with different engineering. Writing "XGBoost is more accurate than gradient boosting" compares a library with the algorithm it implements. Second slip: "K independent trees per round" means independent within the round under the diagonal approximation; the rounds stay sequential, so multi-class boosting is K times the work and parallelism only reduces elapsed time.

Key insight. Multi-class boosting is K binary boostings sharing one softmax, decoupled within a round by dropping the off-diagonal Hessian terms.

How the professor asks it

Q. *Why does multi-class XGBoost fit K trees per round, and in what sense are they independent?*
A. Each class has its own logit F_k and its own gradient $g_k = \hat{p}_k - y_k$, so each needs its own additive correction, hence one tree per class. The full softmax Hessian couples the classes through $-\hat{p}_k \hat{p}_r$, which would make the K leaf values of a round depend on each other; dropping those terms makes the approximate objective a sum of K separate binary-style objectives, so the K trees can be fitted separately and in parallel. The coupling returns when the softmax is recomputed jointly before the next round; the rounds are sequential.

Q. *A report says "we compared four boosting algorithms: AdaBoost, XGBoost, LightGBM and CatBoost". What is wrong with the sentence?*
A. Three of the four are libraries implementing the same algorithm, gradient-boosted trees, with different engineering (histogram splits, leaf-wise growth, ordered categorical encoding); AdaBoost is a different algorithm that reweights observations and combines weak classifiers by weighted vote. The comparison mixes an algorithm with implementations; differences among the three libraries are about speed, defaults and categorical handling, not about the boosting principle.

11 Bagging versus boosting, and when ensembles do not help

Bagging and random forests versus boosting

Question	Bagging / random forest	Boosting
training	parallel randomised fits	sequential corrections
main mechanism	average unstable predictions (variance)	reduce the current loss (mostly bias)
typical trees	deep or moderately deep	shallow (depth 2 to 6)
diversity source	bootstrap rows, feature subsets	changing residuals or gradients
main controls	tree strength versus correlation (m_{try}, leaf size), M until plateau	η, depth, rounds by early stopping, λ, γ
noise sensitivity	generally robust	can chase difficult or mislabelled cases
free validation	OOB error	none; hold out or cross-validate
parallelisation	natural across trees	limited by sequential dependence

These are tendencies: either method can affect both bias and variance.

Bagging against boosting on noisy labels

Setup. Flow data; randomly flip 10% of the training labels (not the validation labels).

Held fixed. The data, the validation set, early stopping for boosting, M on the plateau for the forest.

What we manipulate. Clean labels versus 10% flipped; forest versus boosted trees.

What we measure. Validation error of each ensemble, and for boosting the validation curve against rounds.

What the plot shows. With clean labels boosting usually edges out the forest. With flipped labels the forest degrades gently: each flipped point affects a third of the trees and is outvoted. Boosting degrades more: a flipped point is a large, persistent residual, every round tries to fit it, the leaf containing it gets an extreme output, and the validation curve turns up earlier and higher. Stronger λ, smaller η and larger minimum child weight soften this.

How to read it. Boosting treats every residual as signal; bagging treats every point as one vote. Noisy or mislabelled data favour averaging; clean data with structured residuals favour boosting.

Start from the source of the error

(1) Is one strong learner *unstable* across training samples? Bag it or use a random forest. (2) Can many small corrections improve a *structured residual pattern*? Gradient-boosted trees. (3) Do credible but different model families make *complementary* errors? Validated voting, averaging or stacking. (4) Is the dominant error caused by *missing information* or *deployment shift*? No ensemble will solve it; fix the data and the evaluation protocol.

Purpose. Choose the method from the diagnosis, not from a favourite.

Impact. Ensembles do not help when all models miss the same information ($\rho \approx 1$), when the models are too similar, when boosting starts fitting noise, outliers or wrong labels, when validation data have been reused so often that the choices overfit them, or when test or deployment data differ from the training data. Extra models also cost computation, memory and system complexity, which Lab 3 measures.

Credit scoring, two failure stories

A bank's churn model is a single deep tree whose predictions change every quarter when retrained; the fix was a random forest, which stabilised the ranking of customers. A fraud model's residuals showed a clear pattern (transactions just under the reporting threshold were systematically under-scored); boosted trees learned the correction in a few rounds. A third team tried both on a model whose main error was that merchant category was unavailable at scoring time; both ensembles reproduced the single model's error, and the only fix was to make the feature available.

Choosing by fashion and blaming the ensemble

"Boosting always wins on tabular data" leads to boosting a model whose error is a label-noise problem (it gets worse) or a shift problem (nothing changes). "A forest is safer" leads to bagging a stable linear model (nothing changes). The inflated quantity is the time spent; the deflated one is the trust in the method. Fix: run the instability experiment and the residual inspection first, then pick.

Key insight. Bag what is unstable, boost what has structured residuals, stack what is complementary, and fix the data when the error is shared.

How the professor asks it

Q. *For each situation say whether bagging, boosting, neither or something else is the right response and why: (a) a deep tree whose boundary changes drastically between bootstrap samples; (b) a model whose residuals are systematically positive for one subgroup; (c) a detector that fails on attacks through a gateway absent from the features; (d) a dataset with* 15% *mislabelled rows.*
A. (a) Bagging or a forest: the error is variance and averaging cancels it. (b) Boosting: the residual pattern is structured, and successive trees can learn the subgroup correction. (c) Neither: the error is shared by every model because the information is missing; add the feature and re-evaluate under a grouped or temporal split. (d) Prefer a forest and clean the labels: boosting chases mislabelled points as if they were signal; if boosting must be used, lower η, raise λ and minimum child weight and stop early.

Q. *Explain why boosting has no out-of-bag error and what you use instead.*
A. Every round fits a tree on the full training set (its residuals), so no observation is left out of any model and there is no set of models that never saw a given point; the OOB construction needs bootstrap omission. Use a held-out validation set or K-fold cross-validation to choose M by early stopping and the other knobs by grid search, and an untouched test set at the end.

12 Stacking

Stacking; the meta-model

Base models $m = 1, \dots, M$ (a logistic regression, a random forest, a boosted ensemble) produce predictions z_m, usually probabilities; a *meta-model* h_γ maps them to the final prediction, $\hat{y} = h_{\hat{\gamma}}(z_1, \dots, z_M)$. For binary classification the common meta-model is a logistic regression on the base probabilities, $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$ [on your sheet], fitted on the true labels and on *out-of-fold* base predictions. Soft voting is the special case $\beta_m = 1/M$ up to the sigmoid.

Purpose. Let the data decide the weights: a large positive β_m trusts model m; a small or negative one down-weights an unhelpful or redundant model.

Impact. The meta-model has only M inputs and is kept simple and regularised so that it cannot overfit; the gain comes from complementary base models, not from a clever combiner.

Stacking with in-sample versus out-of-fold base predictions

Setup. Three base models including a deep random forest; a logistic meta-model.

Held fixed. The base models, the data, the validation set.

What we manipulate. How the meta-model's training inputs z are produced: (A) each base model predicts the *same* rows it was trained on; (B) $K = 5$ folds, each base model fitted on four folds and predicting the fifth, so that every row's z comes from a model that never saw it.

What we measure. The fitted β, the meta-model's training loss, and its validation loss.

What the plot shows. In (A) the forest's z equals the label almost exactly on its own training rows, so β_{forest} is huge and the others are near 0; training loss is near 0; validation loss is *worse* than the forest alone. In (B) the weights are moderate and spread, training loss is realistic, validation loss is below every base model.

How to read it. (A) is leakage of training information into the combination rule: the meta-model learns

to trust whichever model overfits most. Out-of-fold predictions are the cure. At prediction time the base models are refitted on all the training data, $(z_1, \ldots, z_M)$ is computed for the new input, and passed through the fitted meta-model.

Training the meta-model on in-sample predictions, and confusing OOF with OOB

The tempting shortcut is to fit the base models once on all data and reuse their predictions on the same rows as meta-features. A deep tree or forest predicts its own training rows almost perfectly, so the meta-model's fit is unrealistically optimistic and it over-weights the most overfitted member; validation performance drops below the best base model. Fix: out-of-fold predictions from an explicit fold split. These are not OOB predictions: OOB is a bootstrap by-product inside a bagged ensemble, out-of-fold is a fold split that works for any base model, including boosting and logistic regression.

Key insight. Stacking is soft voting with learned weights, and it is honest only if the weights are learned from predictions the base models made on rows they never saw.

How the professor asks it

Q. *Describe the training and the prediction phases of a stacked classifier and explain the role of out-of-fold predictions.*
A. Training: split the training set into K folds; for each fold fit every base model on the other folds and predict the held-out fold; join the held-out predictions so every row has $z_m^{(i)}$ from a model that never saw it; fit the meta-model (regularised logistic regression) on $(z^{(i)}, y^{(i)})$. Prediction: refit each base model on all the training data, compute $(z_1, \ldots, z_M)$ for the new input, apply the meta-model. Out-of-fold predictions keep the meta-model from learning "trust the model that memorised its training rows", which in-sample predictions would teach it.

Q. *Give a counterexample showing that stacking can be worse than its best base model, and the condition under which it is better.*
A. Worse: train the meta-model on in-sample predictions with a deep forest among the bases; β_{forest} dominates, the stack reproduces the forest's overfitting and loses to a well-regularised logistic base on validation. Better: when the bases make complementary errors (a linear model good on the bulk, a forest good on interactions) and the meta-weights are fitted on out-of-fold predictions with regularisation, the learned weighting beats each base and beats equal-weight soft voting.

13 Mixture of experts

Mixture of experts; gate; routing

Experts $p_m(y \mid x)$ are combined by a *gating model* $g_m(x) \geq 0$, $\sum_m g_m(x) = 1$: $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$ [on your sheet]. A linear gate scores expert m by $a_m(x) = \beta_m^\top x + b_m$ (a small network for nonlinear routing) and normalises with a softmax, $g = \text{softmax}(a(x))$ [on your sheet]. *Soft routing* uses all experts with their weights; *top-k routing* evaluates only the k experts with the largest gate scores. Gate and experts are trained *jointly* by minimising the negative log-likelihood of the mixture, $-\sum_i \log \left[\sum_m g_m(x^{(i)}) p_m(y^{(i)} \mid x^{(i)}) \right]$ [on your sheet].

Purpose. Let different experts own different regions of the input space, with the regions discovered during training rather than specified in advance.

Impact. Unlike stacking, where β_m is the same for every input, here the weights change with x; top-k routing is also a computational device (conditional computation): with many large experts and $k = 2$, only two are evaluated per input, which is how very large language and vision models use mixtures of experts.

Watching experts specialise, and collapse

Setup. Two-feature flow data with three visibly different attack regimes; three small experts and a linear gate.

Held fixed. The architecture, the loss.

What we manipulate. Training with and without a load-balancing penalty on the gate.

What we measure. Over epochs: the share of inputs routed mainly to each expert, and each expert's validation loss on its own region.

What the plot shows. Without the penalty: one expert's share climbs to nearly 100% while the others idle, the "mixture" degenerates to a single model. With it: the three shares settle near a third each, the gate map shows three regions matching the regimes, and the mixture's validation loss is below any single expert.

How to read it. This is *expert collapse*: an expert that receives more gate weight early learns faster, gets better, attracts still more inputs, and the rich get richer. Load-balancing terms encourage the gate to spread its weight.

Stacking versus mixture of experts

	Stacking	Mixture of experts
weights	global β_m, same for every x	$g_m(x)$, input-dependent
training	base models first, meta-model after, on out-of-fold predictions	gate and experts jointly on the mixture likelihood
specialisation	none; one global trust level per model	experts own regions discovered in training
risk	leakage through in-sample predictions	expert collapse
compute	all bases evaluated	top-k routing evaluates few experts

Key insight. A mixture of experts learns *who* to trust *where*; without load balancing it learns to trust one expert everywhere.

How the professor asks it

Q. *What distinguishes a mixture of experts from stacking, and what is expert collapse?*
A. Stacking learns one global combination $h(z_1, \ldots, z_M)$ after the bases are fitted; a mixture learns input-dependent weights $g_m(x)$ jointly with the experts, so experts can specialise by region of feature space. Expert collapse: an expert favoured early improves faster, attracts more weight, and ends up handling almost all inputs while the others stay untrained; a load-balancing penalty on the gate counters it. Consequence: the mixture needs joint training and balancing, where stacking only needs out-of-fold predictions.

14 Where ensembles are used, and Lab 3

Method, application, reason

Method	Representative applications	Why it is used
Voting / averaging	medical prediction, sensor fusion, deep-network ensembles	combines different models; more stable predictions
Random forest	intrusion detection, remote sensing, bioinformatics, tabular problems	strong baseline with nonlinearity, interactions, little preprocessing
Gradient-boosted trees	credit scoring, fraud detection, churn, ranking on tabular data	high accuracy from successive corrections
Stacking	data-science competitions, systems combining model families	learns when one model should complement another
Mixture of experts	large language and vision models, multi-domain prediction	specialisation and conditional computation

Lab 3 at a glance

Tree ensembles on the cybersecurity flow data of Labs 1 and 2, one fixed stratified split and identical rows for every model, so that differences reflect models and not data. Binary classification: bagged trees and a random forest, asking how much aggregation stabilises the model. Boosting: gradient boosting and XGBoost, asking how rate, depth and rounds interact. Model comparison against the best Lab 2 models, asking which tree errors are corrected. Multiclass: which attack classes remain difficult. Port information: with and without destination port. Efficiency: fit time, prediction time and model size, asking what higher accuracy costs.

How the professor asks it

Q. *Why does Lab 3 insist on the same fixed stratified split and the same rows for every model?*
A. So that a difference in validation or test score can be attributed to the model and not to the sample: with different rows, a forest and a boosted model would be scored on different attacks and the comparison would mix model variance with data variance. Stratification keeps the rare attack classes present in every subset. The same discipline applies to OOB versus validation comparisons: a forest's OOB error and a boosted model's validation error are not on the same rows and should not be compared directly.

Coverage of the textbook chapter

Every item of the textbook chapter, and where it is treated here.

- Why combine: wisdom of the crowd, instability, three families: §1.
- Averaging, hard and soft voting, 0.64 example, calibration gotcha: §2.
- Theorem: variance of an average of correlated errors; $0.232\sigma^2$; bias not removed: §2.
- Decision tree definition, optimal leaf constant, Gini, greedy split by hand, why trees in ensembles: §3.
- Bootstrap sample, count table, out-of-bag fraction theorem, OOB prediction and error, tune the ensemble not the trees: §4.
- Random forest definition, probability a feature is offered, strength versus correlation, controls, forest-level selection: §5.
- MDI credit and forest-level MDI, bias gotcha; permutation importance, masking and causality gotcha: §6.
- Additive model, residual-fitting theorem, gradient boosting algorithm, one round by hand, deep versus shallow, the triad and early stopping: §7.
- Logit boosting, g and h theorem, classification algorithm and line search, Newton leaf value with λ, worked leaf: §8.
- XGBoost objective, weighted-regression theorem, leaf value and gain theorem, worked gain, four mechanisms, further controls: §9.
- Multi-class derivatives, one multi-class round, AdaBoost and libraries gotcha: §10.
- Bagging versus boosting table, start from the source of error, when ensembles do not help: §11.

- Stacking definition, out-of-fold technique, in-sample gotcha: §12.
- Mixture of experts, joint training, expert collapse: §13.
- Applications table, Lab 3, notes on the slides (sign conventions, λJ versus γQ): §14, §8, §9.

The ten mistakes of this chapter

1. Expecting an ensemble to remove a *shared* error (missing feature, shift): averaging cancels only what differs.
2. Quoting σ^2/M for correlated models; the floor is $\rho\sigma^2$ and more trees cannot pass it.
3. Soft-voting with a miscalibrated member, which then decides every vote; or hard-voting away a deserved confidence.
4. Reporting the OOB error as a test result, or selecting on it repeatedly and reporting the minimum.
5. Tuning each tree of a forest separately, or giving the trees all features to make them "stronger".
6. Reading feature importance as causality, or ignoring that correlated twins mask each other and that MDI favours many split points.
7. Choosing the number of boosting rounds by training loss, or comparing learning rates at a fixed M.
8. Sign slips between $r = y - \hat{p}$ and $g = \hat{p} - y$; forgetting that $h \to 0$ makes Newton steps explode and that λ and minimum child weight exist for that.
9. Saying λ discourages splits: λ shrinks outputs, γ taxes leaves; calling XGBoost, LightGBM, CatBoost and AdaBoost "four algorithms".
10. Training a stacking meta-model on in-sample base predictions; confusing out-of-fold with out-of-bag; expecting an OOB error from boosting.

Chapter 5 in ten lines

1. A flexible single fit has high variance; averaging competent models with different errors cancels part of it; shared errors survive.
2. $\mathrm{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$: independent errors give σ^2/M, correlation leaves a floor $\rho\sigma^2$; every trick lowers ρ.
3. Trees: boxes with one constant each (mean, median, majority), greedy Gini splits, flexible and unstable, the ideal ensemble member.
4. Bagging: bootstrap samples (63.2% in, 36.8% out), average or vote; OOB error is free internal validation, not a test set.
5. Random forest: m_{try} random candidates per split decorrelate the trees; weaker trees, stronger forest; tune at forest level; add trees until the OOB plateau.
6. Importance: MDI (training credits, biased to many thresholds) and permutation (error increase on OOB rows); masking by correlation; never causality.
7. Boosting: $F_m = F_{m-1} + \eta T_m$, each tree fitted to the negative gradient; squared error gives residuals; M, η, depth are coupled and chosen with early stopping on validation.
8. Classification boosts the logit: $g = \hat{p} - y$, $h = \hat{p}(1-\hat{p})$; Newton leaf $\sum(-g)/(\sum h + \lambda)$; small $\sum h$ explodes without λ.
9. XGBoost: one regularised second-order objective for splits and outputs; target $-g/h$ with weight h; $w^* = -G/(H+\lambda)$; gain minus γ per leaf.
10. Stacking learns global weights from out-of-fold predictions; a mixture of experts learns input-dependent weights jointly with the experts; bag the unstable, boost the structured residual, and fix the data when the error is shared.